

同济大学计算机系

数字逻辑课程实验报告



学 号 2351520

姓 名 毛星博

专 业 计算机科学与技术

授课老师 张冬冬

6.8_1 ram

一、实验内容

1) RAM

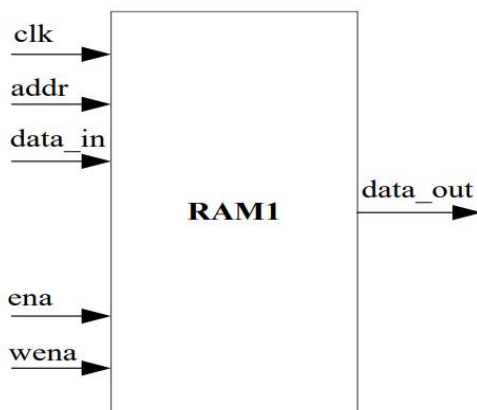
半导体随机读写存储器，简称 RAM，它是数字计算机和其他数字系统的重要存储部件，可存放大量的数据。图 6.25 所示为 RAM 的逻辑结构图，其主体是存储矩阵，另有地址译码器和读写控制电路两大部分。读写控制电路中加有片选控制和输入输出缓冲器等，以便组成双向 I/O 数据线。



图 6.25 RAM 的逻辑结构图

RAM 有三组信号线：

- 地址线：单向，传送地址码（二进制数），以便按地址码访问存储单元；
- 数据线：双向，将数据码（二进制数）送入存储矩阵或从存储矩阵读出；
- 读/写命令线：单向控制线，分时发送这两个命令，要保证读时不写，写时不读。



实验 1 要求 ram 的数据输入和输出分为两条数据线。

二、硬件逻辑图

无

三、模块建模

（该部分要求对实验中建模的所有模块进行功能描述，并列出各模块建模的

verilog 代码)

```
module ram (  
input clk, //存储器时钟信号, 上升沿时向 ram 内部写入数据  
input ena, //存储器有效信号, 高电平时存储器才运行, 否则输出 z  
input wena, //存储器读写有效信号, 高电平为写有效, 低电平为读有效, 与 ena 同时有效时  
才可对存储器进行读写  
input [4:0] addr, //输入地址, 指定数据读写的地址  
input [31:0] data_in, //存储器写入的数据, 在 clk 上升沿时被写入  
output reg [31:0] data_out //存储器读出的数据  
);  
reg [31:0] ram [31:0];  
  
// 写操作在时钟上升沿进行,读操作不依赖时钟  
always @(posedge clk or negedge wena) begin  
    if(ena) begin  
        if(wena)  
            ram[addr] <= data_in;  
        else  
            data_out <= ram[addr];  
    end  
    else  
        data_out <= 32'hzzzz_zzzz;  
end  
  
endmodule
```

四、测试模块建模

(要求列写各建模模块的 test bench 模块代码)

```
module ram_tb();  
    reg clk, ena, wena;  
    reg [4:0] addr;  
    reg [31:0] data_in;  
    wire [31:0] data_out;  
  
    ram uut(clk,ena,wena,addr,data_in,data_out);  
  
    initial begin  
        clk = 1;  
        ena = 1;  
        wena = 0;  
        addr = 0;
```

```

        data_in = 32'hffff_ffff;
end

always
    #20 clk = ~clk;

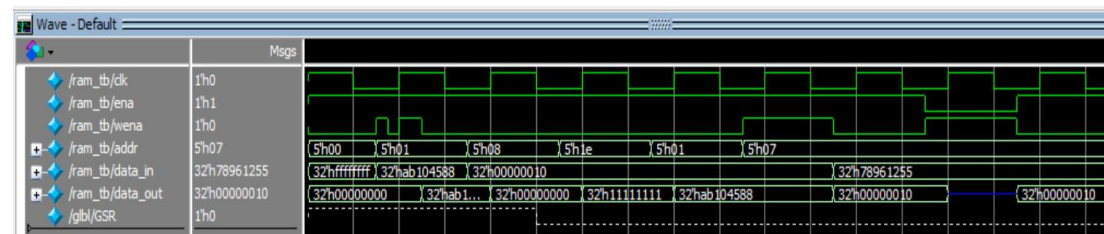
initial begin
    #30 wena = 1;
        addr = 1; data_in = 32'h010_4588;
    #5  wena = 0;
    #5  wena = 1;
    #10 wena = 0;
    #20 wena = 0;
        addr = 8; data_in = 32'h0000_0010;
    #40 addr = 30;
    #40 wena = 0;
        addr = 1;
    #40 addr = 7;
        wena = 1;
    #40 wena = 0;
        addr = 7; data_in = 32'h7896_1255;
    #40 wena = 1;
        ena = 0;
    #40 ena = 1;
        wena = 0;
end
endmodule

```

五、实验结果

（该部分可截图说明，要求 logisim 逻辑验证图、modelsim 仿真波形图、以及下板后的实验结果贴图（实验步骤中没有下板要求的实验，不需要下板贴图））

Modelsim 仿真波形图：



实验结果符合预期，试验成功。

6.8_2 ram2

一、实验内容

1) RAM

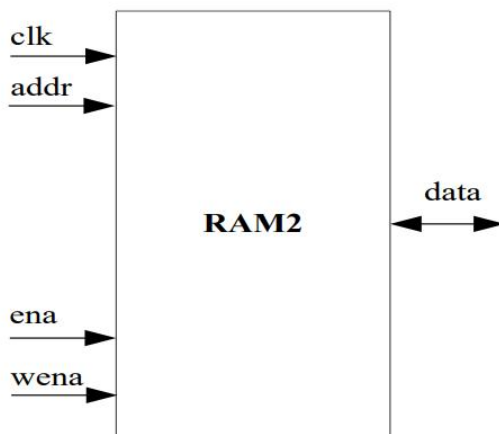
半导体随机读写存储器，简称 RAM，它是数字计算机和其他数字系统的重要存储部件，可存放大量的数据。图 6.25 所示为 RAM 的逻辑结构图，其主体是存储矩阵，另有地址译码器和读写控制电路两大部分。读写控制电路中有片选控制和输入输出缓冲器等，以便组成双向 I/O 数据线。



图 6.25 RAM 的逻辑结构图

RAM 有三组信号线：

- 地址线：单向，传送地址码（二进制数），以便按地址码访问存储单元；
- 数据线：双向，将数据码（二进制数）送入存储矩阵或从存储矩阵读出；
- 读/写命令线：单向控制线，分时发送这两个命令，要保证读时不写，写时不读。



本实验要求 ram 的数据输入和输出使用同一条数据线。

二、硬件逻辑图

无

三、 模块建模

```
module ram2(
    input clk,
    input ena,
    input wena,
    input [4:0] addr,
    inout [31:0] data
);
    reg [31:0] ram [0:31];
    reg [31:0] temp;
    integer i;

    initial begin
        for (i = 0; i < 16; i = i + 1) begin
            ram[i] = 32'h0000_0000;
        end
        for (i = 16; i < 32; i = i + 1) begin
            ram[i] = 32'h1111_1111;
        end
    end

    always @(posedge clk) begin
        if (ena && wena) begin
            ram[addr] <= data; // 写操作
        end
    end

    always @(negedge wena) begin
        if (ena && !wena) begin
            temp <= ram[addr]; // 读操作
        end
    end

    assign data = (ena && !wena) ? temp : 32'bz; // 在读操作时输出数据，否则高阻态

endmodule
```

四、测试模块建模

```
module ram2_tb();
    reg clk;
    reg ena;
    reg wena;
    reg [4:0] addr;
    wire [31:0] data;
    reg [31:0] data_in;

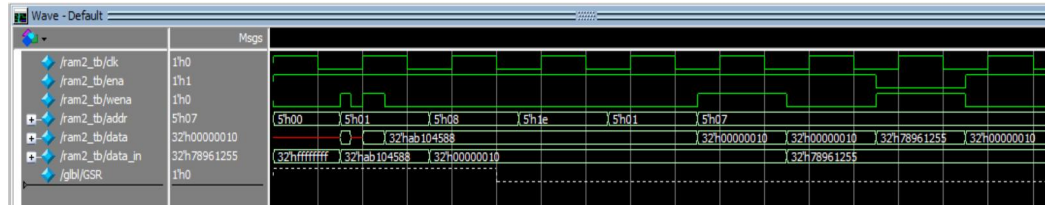
    ram2 uut(clk, ena, wena, addr, data);

    initial begin
        clk = 1;
        ena = 1;
        wena = 0;
        addr = 0;
        data_in = 32'hffff_ffff;
    end
    always
        #20 clk = ~clk;

    assign data = wena ? data_in : 32'hzzzz_zzzz;
    initial begin
        #30 wena = 1;
        addr = 1; data_in = 32'h010_4588;
        #5  wena = 0;
        #5  wena = 1;
        #10 wena = 0;
        #20 wena = 0;
        addr = 8; data_in = 32'h0000_0010;
        #40 addr = 30;
        #40 wena = 0;
        addr = 1;
        #40 addr = 7;
        wena = 1;
        #40 wena = 0;
        addr = 7; data_in = 32'h7896_1255;
        #40 wena = 1;
        ena = 0;
        #40 ena = 1;
        wena = 0;
    end
endmodule
```

五、实验结果

Modelsim 仿真波形图

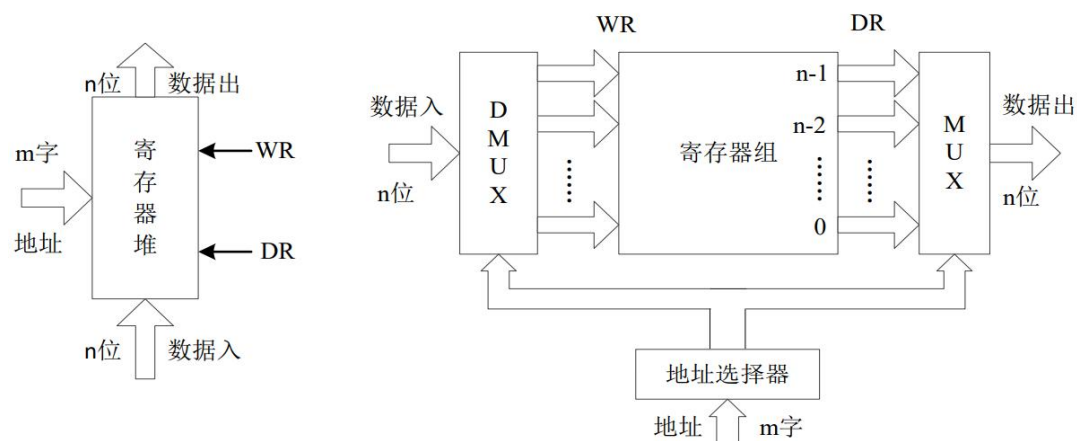


经验证输出符合预期，实验成功。

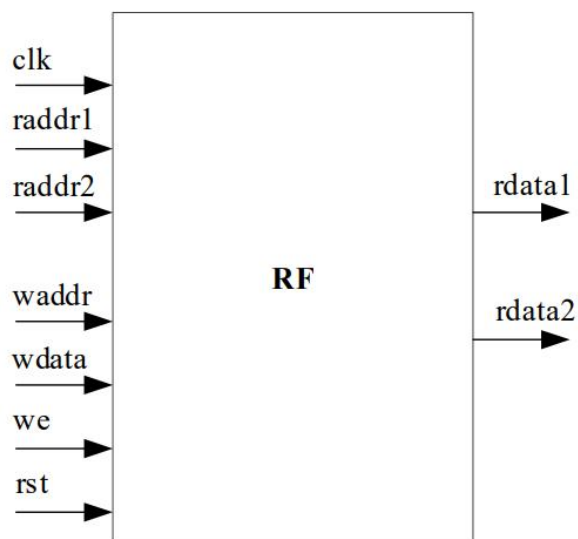
6.8_3 寄存器堆 (regfiles)

一、实验内容

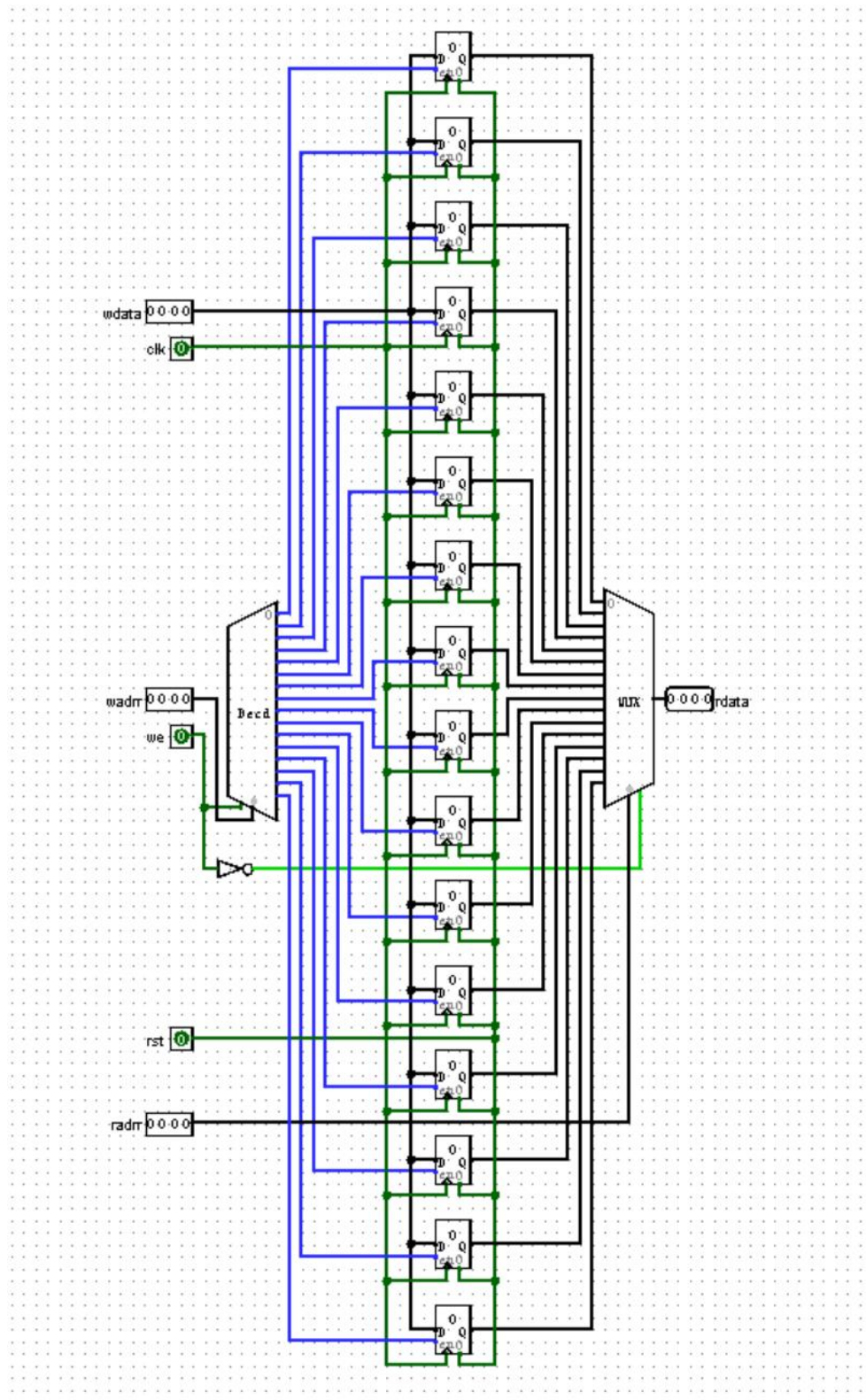
一个寄存器是由 n 个触发器或锁存器按并行方式输入且并行方式输出连接而成。它只能记忆 1 个字，1 个字的长度等于 n 个比特。当需要记忆多个字时，一个寄存器就不够用了，在这种情况下，需要使用由多个寄存器组的寄存器堆。它由寄存器组、地址译码器、多路选择器 MUX 及多路分配器 DMUX 等部分组成。向寄存器写数据或读数据，必须先给出寄存器的地址编号。写数据时，控制信号 WR 有效，待写入的数据经 DMUX 送到地址给定的某个寄存器。读数据时，控制信号 RD 有效，由地址给定的某个寄存器的数据内容经多路开关 MUX 送出。由于读/写工作是分时进行的，所以寄存器组在逻辑上能满足写数据或读数据的需要。



本实验要完成 32 位寄存器堆的实现。



二、硬件逻辑图



三、模块建模

```
module Regfiles(
input clk, //寄存器组时钟信号，下降沿写入数据
input rst, //异步复位信号，高电平时全部寄存器置零
input we, //寄存器读写有效信号，高电平时允许寄存器写入数据，低电平时允许寄存器读出数据
input [4:0] raddr1, //所需读取的寄存器的地址
input [4:0] raddr2, //所需读取的寄存器的地址
input [4:0] waddr, //写寄存器的地址
input [31:0] wdata, //写寄存器数据，数据在 clk 下降沿时被写入
output reg [31:0] rdata1, //raddr1 所对应寄存器的输出数据
output reg [31:0] rdata2 //raddr2 所对应寄存器的输出数据
);

reg [31:0] regfile [0:31]; //寄存器组，共 32 个寄存器，每个寄存器 32 位
integer i;

always @(negedge clk,posedge rst)
begin
    if (rst) //异步复位信号为高电平时，全部寄存器置零
        for (i = 0; i < 32; i = i + 1)
            regfile[i] = 32'b0;
    else if (we) //寄存器读写有效信号为高电平时，允许寄存器写入数据
        regfile[waddr] <= wdata; //写寄存器数据，数据在 clk 下降沿时被写入
end

always @(*)
begin
    rdata1 <= we ? 32'bz: regfile[raddr1]; //raddr1 所对应寄存器的输出数据
    rdata2 <= we ? 32'bz: regfile[raddr2]; //raddr2 所对应寄存器的输出数据=
end

Endmodule
```

四、测试模块建模

```
module Regfiles_tb();
    reg clk, rst, we;
    reg [4:0] raddr1;
    reg [4:0] raddr2;
    reg [4:0] waddr;
    reg [31:0] wdata;
    wire [31:0] rdata1;
```

```

wire [31:0] rdata2;

always
    #20 clk = ~clk;

Regfiles uul(clk, rst, we, raddr1, raddr2, waddr, wdata, rdata1, rdata2);

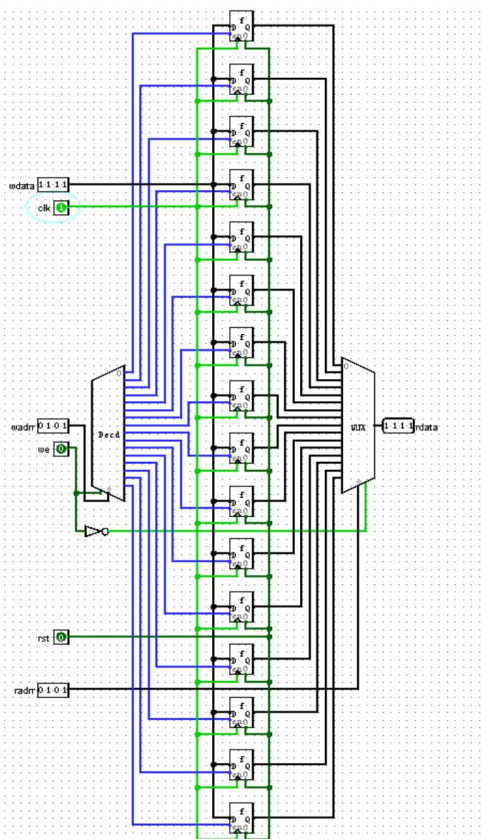
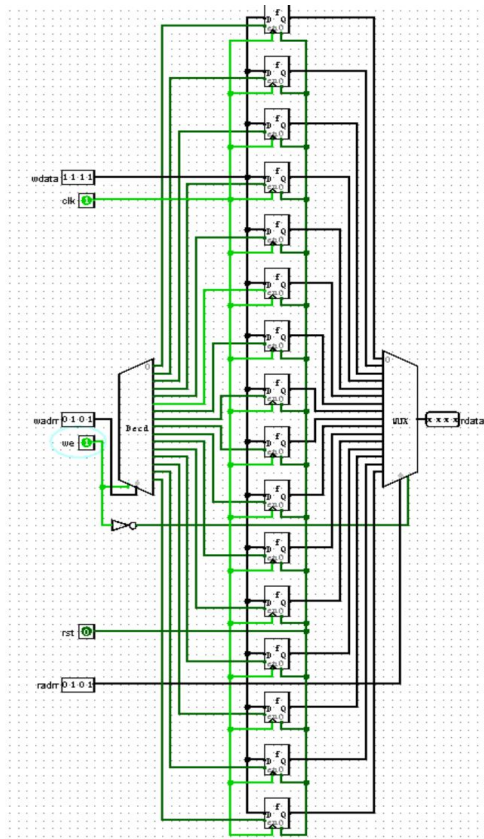
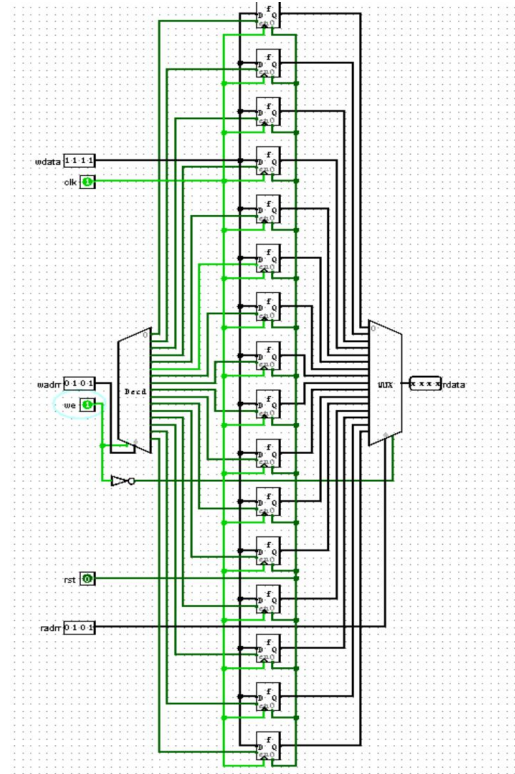
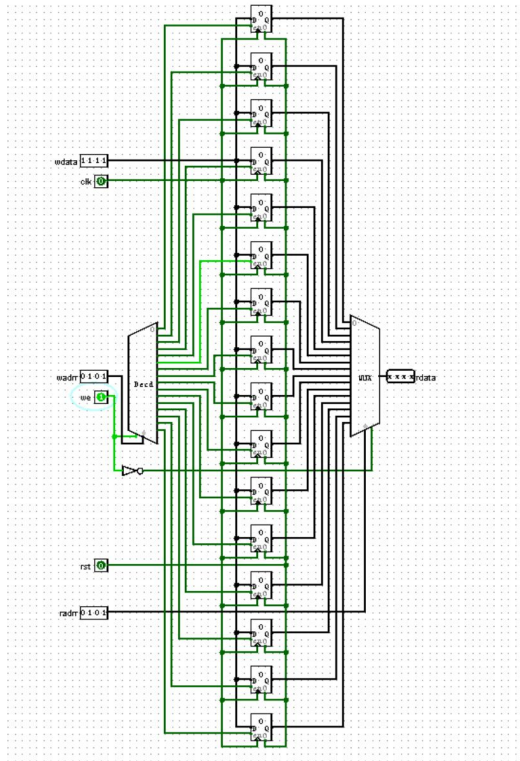
initial begin
    clk = 0; rst = 1; we = 1;
    raddr1 = 0; raddr2 = 0;
    waddr = 0;
    wdata = 32'h0a2e_5486;
    #10
    rst = 0;
    #10
    #15 waddr = 10; wdata = 32'h5894_a5ff;
    #20 waddr = 7; wdata = 32'hffff_ffff;
    #20 waddr = 1; wdata = 32'h0000_0000;

    #20 we = 0;
    #20 raddr1 = 1; raddr2 = 7;
    #20 raddr1 = 10; raddr2 = 0;
    #20 raddr1 = 5; raddr2 = 4;
    rst = 1;
    #20 raddr1 = 5; raddr2 = 4;
end
endmodule

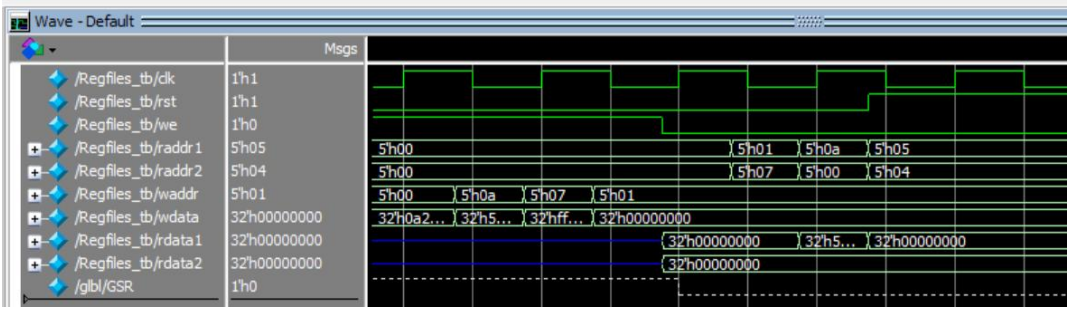
```

五、实验结果

Logicsim 逻辑验证：



Modelsim 仿真波形图：



实验结果与逻辑相符合，实验成功。